Compte Rendu final de Projet de Recherche

Pierre Baumann, Ivan Mulot-Radojcic, Damien Assire

30 avril 2024

Résumé

Abstract

Table des matières

1 Introduction **1**

2 Calcul de la mesure **1**
2.1 Ancien algorithme 1
2.2 Nouvel algorithme 1
2.2.1 Création d'une structure de données adaptée 2
2.2.2 Algorithme de calcul de la mesure 3
2.2.3 Optimisations faites 3
2.2.3.1 Calcul de la distance entre le noeud de départ et tous les autres noeuds 3
2.2.3.2 Prise en compte de tous les sous-chemins 3
2.2.3.3 Prédiction des chemins avec le plus d'étapes 3
2.2.3.4 Regarder ses voisins 3
2.2.3.5 Se rapporter à un parcours de graphe classique quand c'est possible . 4
2.2.3.6 Calcul du non-attqué 4
2.2.4 Complexité de l'algorithme 4

sections

1 Introduction

Introduct

2 Calcul de la mesure

2.1 Ancien algorithme

Ancien algorithme

2.2 Nouvel algorithme

Le problème des plus courts chemins est déjà assez complexe dans un graphe classique, mais il devient encore plus difficile dans un graphe temporel, où les arêtes peuvent être bloquées à certains moments. Nous sommes d'abord partis sur une adaptation de l'algorithme de Floyd-Warshall, car nous savons que cet algorithme marche tout type de graphe, notamment des graphes étranges comme les ceux avec des arêtes à poids négatifs.

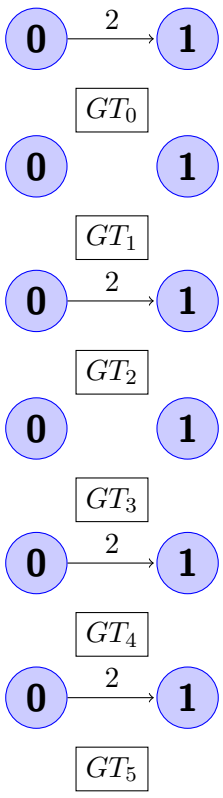
Le problème des calculs de chemin dans les graphes temporels

```
TemporalFloydWarshall(graphe temporel GT, DistanceMatrixes dma)
  InitTFW(GT, dma)
  for time in {dma.NumMatrixes .. 0} do
    for k in {0 .. |GT.V| } do
      for u in {0 .. |GT.V| } do
        for v in {0 .. |GT.V| } do
          distUV ← dma.matrix[t_i][u][v]
          distUK ← dma.matrix[t_i][u][k]
          distKV ← dma.matrix[t_i + distUK][k][v]
          if distUV > distUK + distKV then
            dma.matrix[t_i][u][v] ← distUK + distKV
          end if
        end for
      end for
    end for
  end for
```

2.2.1 Création d'une structure de données adaptée

Le problème des autres algorithmes classiques de graphes est qu'ils ne peuvent pas voir dans le futur, et donc ne peuvent pas prendre en compte les liens bloqués. Cependannt, nous avons pensé à créer des liens fantômes qui remplacent les liens supprimés, équivalents à la distance réelle plus l'attente nécessaire. Cela va permettre de ne pas avoir besoin de regarder dans le futur pour calculer les chemins, tout en calculant quand même la vraie distance minimale, et ainsi rendre beaucoup plus de parcours de graphes adaptables temporellement, notamment Dijkstra, que nous avons utilisé pour notre implémentation.

Mettre ca en verti-cal et rendre ça moins moche et finir exemple



2.2.2 Algorithme de calcul de la mesure

2.2.3 Optimisations faites

L'algorithme de Dijkstra est donc mis en place et adapté pour les graphes temporels, mais il reste encore des optimisations à faire. En effet, l'algorithme de Dijkstra est souvent utilisé pour calculer le plus court chemin entre deux noeuds, mais il calcule en réalité beaucoup plus de plus courts chemins. Donc nous avons modifié l'algorithme pour qu'il écrive directement dans la matrice de distances, puis le but va être d'écrire le plus de distances possibles en une seule fois.

2.2.3.1 Calcul de la distance entre le noeud de départ et tous les autres noeuds

Pour connaître la distance entre le noeud de départ et le noeud d'arrivée, Dijkstra va explorer beaucoup de noeuds candidats. Donc, pour chaque noeud candidat à une distance d du noeud de départ qu'il sort du tas, on va noter cette distance dans la matrice de distances.

2.2.3.2 Prise en compte de tous les sous-chemins

Soit un chemin calculé 0->1->2->3, on a un chemin trivial à noter entre 0 et 3, mais on a aussi beaucoup de sous-chemins qui sont calculés. Ici, on a les chemins 0->1->2, 1->2->3, 0->1, 1->2, 2->3. Et en général, pour un chemin de taille n, on a $n(n-1)/2$ sous-chemins qu'on peut noter dans la matrice de distances.

Nous avons donc créer une structure de données qui va permettre de rapidement stocker tous les sous-chemins à partir des pas faits dans le chemin. Cette structure ressemble à un arbre de segments, mais en structure pyramidale. Pour l'exemple précédent, notre pyramide ressemblerait à ça : 0->3 0->1, 1->3 0->1, 1->2, 2->3

Faire un dessin de la pyramide

Les éléments 0->1, 1->2, 2->3 sont les briques de la pyramide, qui sont l'équivalent des feuilles de l'arbre de segments. Dans notre pyramide, chaque élément va stocker le noeud de départ, le noeud d'arrivée, la distance entre ces deux noeuds, et le temps nécessaire pour arriver au noeud de départ. Ensuite, pour chaque étage de la pyramide, on va regarder notre fils gauche et notre fils droit, ainsi que notre petit-fils commun. Le noeud de départ sera celui du fils gauche, le noeud d'arrivée celui du fils droit, la distance sera la somme des distances des deux fils moins la distance du petit-fils (partie commune comptée deux fois sinon), et le temps nécessaire pour arriver au noeud de départ sera celui du fils gauche. S'il n'y a pas de petit-fils, alors la distance commune entre les 2 est comptée.

En les propageant de bas en haut, on va noter tous les sous-chemins possibles dans la matrice de distances, et donc éviter de les recalculer plus tard.

2.2.3.3 Prédiction des chemins avec le plus d'étapes

Cette optimisation va de pair avec la précédente, en faisant des benchmarks, nous avons remarqué que parcourir la boucle j à l'envers permettait de gagner du temps. On suppose que c'est dû au fait que les noeuds avec des indices éloignés soient aussi éloignés dans le graphe, donc que le chemin entre ces deux noeuds ai plus d'étapes, et que donc on va écrire plus de sous-chemins dans la matrice de distances qui seraint écrasés plus tard si on avait parcouru la boucle dans le sens normal.

2.2.3.4 Regarder ses voisins

La distance de u à v est le minimum entre la distance de u à k plus la distance de k à v pour chaque noeud k tel que l'arête $u->k$ et l'arête $k->v$ existent. Donc si nous connaissons toutes ces distances $u->k$, alors nous pouvons calculer la distance $u->v$ sans avoir besoin de reparcourir le graphe.

Nous avons utilisé cette propriété pour optimiser notre algorithme. Tout d'abord, nous créons le graphe inverse, qui pour chaque arête $u->v$ du graphe original va créer une arête $v->u$.

Ensuite, pour avoir la distance entre u et v, nous cherchons tous les noeuds k tels que l'arête $k->v$ existe grâce au graphe inverse. Pour chaque noeud k, si la distance est calculée, alors nous pouvons directement la récupérer, sinon nous la calculons. Ensuite, nous prenons le minimum de toutes ces

distances ($u->k+k->v$) pour avoir la distance $u->v$. C'est très utile pour les noeuds qu'on ne peut atteindre que par un seul autre noeud, car ça évite de refaire un parcours avec juste un noeud en plus.

2.2.3.5 Se rapporter à un parcours de graphe classique quand c'est possible

Attaques statiques

2.2.3.6 Calcul du non-attqué

blehhhhh

2.2.4 Complexité de l'algorithme

est ce que je parle du cache et de la edges map ?

Donc même si la compléxité pire cas de faire des Dijkstra pour chaque noeud est en $O(n^3 \log n * t)$, la complexité amortie est bien meilleure. La calculer réellement serait extrêmement complexe et dépend beaucoup de la topologie du graphe, mais sur des villes réelles, nous avons mesurer l'évolution du temps de calcul en fonction du nombre de noeuds, et nous avons obtenu cette courbe : qui peut être modélisée par une fonction en $O(???????)$.

Courbe de la complexité

Calcul de la mesure

Références